

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Timery i zegar czasu rzeczywistego

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 7

Studia dzienne

Informatyka

Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salauyou

Data: 21.04.2022

Zadanie 1

Stwórz projekt counter_modulo_k N-bitowego licznika modulo k. Licznik powinien liczyć od 0 do k-1. Dodatkowe wyjście **rollover** musi przyjmować wartość 1 w takcie zegara, gdy wartość licznika wynosi k-1. Jako parametr należy użyć wartości k, a liczbę bitów N należy wyznaczyć za pomocą funkcji stałej **clogb2**. Ponadto licznik musi mieć asynchroniczne wejście zerujące aclr i umożliwić włączenie sygnałem **enable**. Zaimplementować projekt na płycie dla k = 20 z następującym przyporządkowaniem pinów:

Wyprowadzenia	Piny
clk	KEY0
aclr	KEY1
enable	KEY2
Q[N-1:0]	LEDR[4:0]
rollover	LEDR9

Kod w Verilog

Moduł N-bitowego licznika modulo k

```
module counter_modulo_k
#(parameter k = 20)( // okres licznika
input clk, aclr, enable,
output reg [N-1:0] Q,
output rollover);

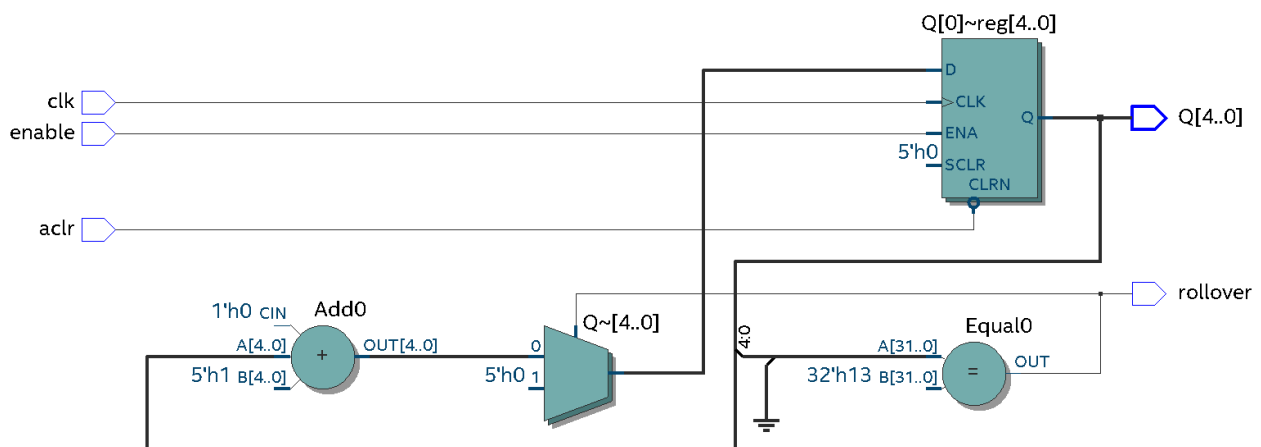
function integer clogb2(input [31:0] v);
for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
v = v >> 1;
endfunction

localparam N = clogb2(k-1); // długość licznika w bitach

always @ (posedge clk, negedge aclr)
if (!aclr) Q <= {N{1'b0}};
else begin
// zanim sprawdzimy, czy Q == k-1 i ewentualnie wyzerujemy licznik, sprawdzamy, czy na wejściu zezwalającym jest 1
if (enable) begin
if (Q == k-1) Q <= {N{1'b0}};
else Q <= Q + 1'b1;
end
else Q <= Q;
end

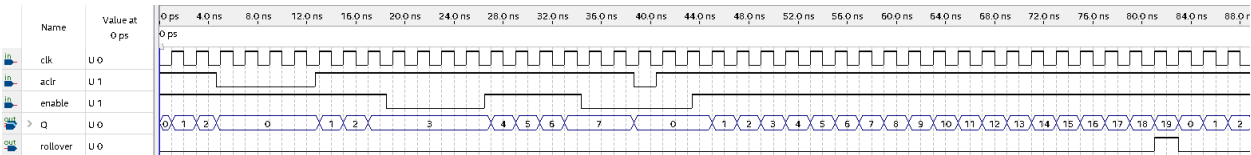
assign rollover = (Q == k-1);
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Przyjmujemy parametr $k = 20$, więc licznik jest modulo 20.



Zadanie 2

Utwórz projekt counter_BCD_3_digits 3-bitowego licznika dziesiętnego, który liczy od 0 do 999 w odstępie 1 sekundy. Aby zsynchronizować projekt, użyj zegara CLOCK_50 o częstotliwości 50 MHz. Wyświetl wartość licznika na ekranach HEX2-0. Aby zresetować licznik, użyj przycisku KEY0. Po osiągnięciu maksymalnej wartości (999) świeci dioda LEDR0.

Wskazówka: Użyj kilku liczników modulo 10. Użyj projektu opóźnienia delay_1_sec, który generuje sygnał zegara z interwałem 1 s na podstawie zegara CLOCK_50.

Kod w Verilog

Moduł 3-cyfrowego licznika dziesiętnego w kodzie BCD

```
module counter_BCD_3_digits(
    input clock,
    input areset,
    output [11:0] bcd, // 3 cyfry BCD po 4 bity
    output max);

    wire c0, c1, c2, c3;
    delay_1_sec delay(clock, c0);
    counter_modulo_k #(10) counter0(c0, areset, 1'b1, bcd[3:0], c1);
    counter_modulo_k #(10) counter1(~c1, areset, 1'b1, bcd[7:4], c2);
    counter_modulo_k #(10) counter2(~c2, areset, 1'b1, bcd[11:8], c3);
    assign max = c1 & c2 & c3;

endmodule
```

Moduł opóźnienia generujący sygnał zegara slow_clock z interwałem 1 sekundy. Zegar CLOCK_50 w płycie DE1-SoC ma częstotliwość 50 MHz, więc w module opóźnienia używamy licznika modulo 50 000 000.

```
module delay_1_sec(
    input clock,
    output slow_clock); // zbocze 0 -> 1 w momencie wyzerowania (przekręcenia) licznika

    wire clock_rollover;
    counter_modulo_k #(50_000_000) counter(.clk(clock), .aclr(1'b1), .enable(1'b1), .rollover(clock_rollover));
    assign slow_clock = ~clock_rollover;

endmodule
```

Moduł N-bitowego licznika modulo k

```
module counter_modulo_k
#(parameter k = 20) ( // okres licznika
    input clk, aclr, enable,
    output reg [N-1:0] Q,
    output rollover);

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

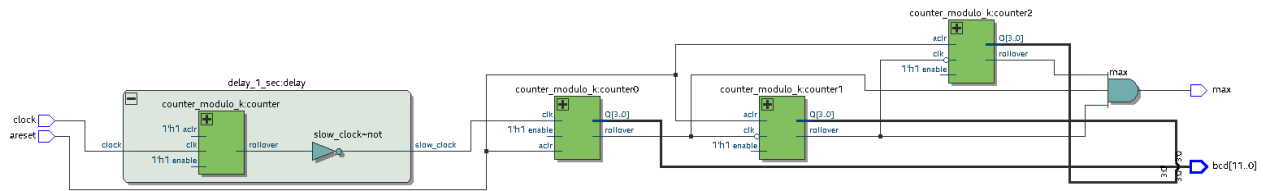
    localparam N = clogb2(k-1); // długość licznika w bitach

    initial Q = {N{1'b0}};
    always @ (posedge clk, negedge aclr)
        if (!aclr) Q <= {N{1'b0}};
        else begin
            // zanim sprawdzimy, czy Q == k-1 i ewentualnie wyzerujemy licznik, sprawdzamy, czy na wejściu zezwalającym jest 1
            if (enable) begin
                if (Q == k-1) Q <= {N{1'b0}};
                else Q <= Q + 1'b1;
            end
            else Q <= Q;
        end

    assign rollover = (Q == k-1);

endmodule
```

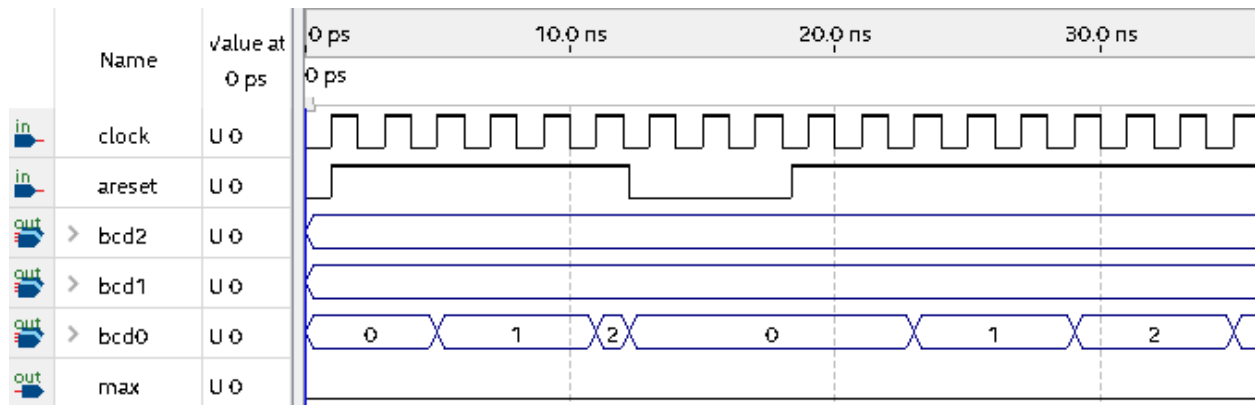
Wyniki syntezy



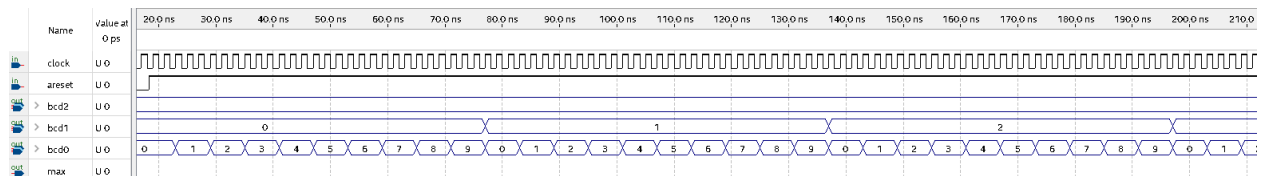
Wyniki symulacji funkcjonalnej

Aby w symulacjach było widać działanie 3-cyfrowego licznika dziesiętnego, parametryzujemy moduł opóźnienia delay_1_sec jako licznik modulo 3. Wówczas wartość „bcd” licznika dziesiętnego zwiększa się o 1 co 3 zbocza narastające zegara „clock”.

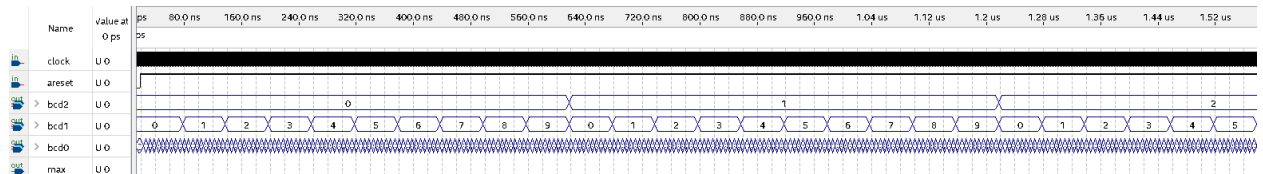
Resetowanie licznika



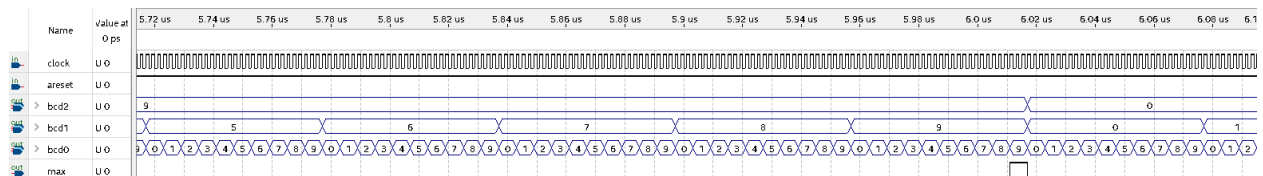
Odciskanie w skali jednostki i dziesiątek



Odciskanie w skali dziesiątek i setek



Signalizowanie maksymalnej wartości 999



Zadanie 3

Utwórz zegar czasu rzeczywistego `real_time_clock`. Minuty (0-59) mają być wyświetlane na wyświetlaczach HEX5-4, sekundy - HEX3-2, setne sekundy - HEX1-0. Zaimplementuj wprowadzanie danych.

Aby wprowadzić setne sekundy, po naciśnięciu przycisku KEY1, użyj przełączników SW7-0.

Aby wprowadzić sekundy, po naciśnięciu przycisku KEY2, użyj przełączników SW7-0.

Aby wprowadzić minuty, po naciśnięciu przycisku KEY3, użyj przełączników SW7-0.

Timer ma zacząć odliczanie po naciśnięciu przycisku KEY0.

Wskazówka: stwórz projekt `delay_10ms`, który generuje sygnał zegarowy z interwałem 10 milisekund (ms). Użyj tego sygnału synchronizacji, aby zsynchronizować najniższy z liczników w module. Użyj wyjścia rollover do synchronizacji każdego następnego licznika modulo. Zwróć uwagę na wartość modulo każdego licznika.

Kod w Verilog

Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty

```
module real_time_clock_on_board(
    input CLOCK_50,
    input [7:0] SW,
    input [3:0] KEY,
    output [6:0] HEX5, HEX4, HEX3, HEX2, HEX1, HEX0);

    wire [7:0] m, s, cs;
    real_time_clock rtc(CLOCK_50, ~KEY[0], ~KEY[3], ~KEY[2], ~KEY[1], SW, m, s, cs);

    decoder_hex_10 d0(cs[3:0], HEX0);
    decoder_hex_10 d1(cs[7:4], HEX1);
    decoder_hex_10 d2(s[3:0], HEX2);
    decoder_hex_10 d3(s[7:4], HEX3);
    decoder_hex_10 d4(m[3:0], HEX4);
    decoder_hex_10 d5(m[7:4], HEX5);

endmodule
```

Moduł dekodera z kodu binarnego na kod wyświetlacza 7-segmentowego

```
module decoder_hex_10(
    input [3:0] binary,
    output reg [6:0] h);

    always @ (*)
        case (binary)
            4'd0: h = 7'b1000000; // 64
            4'd1: h = 7'b1111001; // 121
            4'd2: h = 7'b0100100; // 36
            4'd3: h = 7'b0110000; // 48
            4'd4: h = 7'b0011001; // 25
            4'd5: h = 7'b0010010; // 18
            4'd6: h = 7'b0000010; // 2
            4'd7: h = 7'b1111000; // 120
            4'd8: h = 7'b0000000; // 0
            4'd9: h = 7'b0010000; // 16
            default: h = 7'b1111111; // 127
        endcase

endmodule
```

Moduł zegara czasu rzeczywistego

```
module real_time_clock(
    input clock, toggle_enable, m_aload, s_aload, cs_aload,
    input [7:0] data,
    // minuty, sekundy, centysekundy; na każdą jednostkę czasu 2 cyfry BCD po 4 bity
    output [7:0] m, s, cs);

    reg enable = 1'b0;
    always @ (posedge toggle_enable)
        enable <= ~enable;

    wire c1, c2, c3, c4, c5;
    wire [3:0] h0, h1, h2, h4;
    wire [2:0] h3, h5;
    assign cs = {h1, h0};
    assign s = {1'b0, h3, h2};
    assign m = {1'b0, h5, h4};

    wire c0;
    delay_10ms delay(clock, c0);

    counter_modulo_k #(10) count_cs0(c0, 1'b1, enable, cs_aload, data[3:0], h0, c1);
    counter_modulo_k #(10) count_cs1(~c1, 1'b1, 1'b1, cs_aload, data[7:4], h1, c2);
    counter_modulo_k #(10) count_s0(~c2, 1'b1, 1'b1, s_aload, data[3:0], h2, c3);
    counter_modulo_k #(6) count_s1(~c3, 1'b1, 1'b1, s_aload, data[6:4], h3, c4);
    counter_modulo_k #(10) count_m0(~c4, 1'b1, 1'b1, m_aload, data[3:0], h4, c5);
    counter_modulo_k #(6) count_m1(~c5, 1'b1, 1'b1, m_aload, data[6:4], h5);

endmodule
```

Moduł opóźnienia generujący sygnał zegara slow_clock z interwałem 10 milisekund. Zegar CLOCK_50 w płycie DE1-SoC ma częstotliwość 50 MHz, więc w module opóźnienia używamy licznika modulo 500 000.

```
module delay_10ms(
    input clock,
    output slow_clock); // zbocze 0 -> 1 w momencie wyzerowania (przekręcenia) licznika

    wire clock_rollover;
    /* 1 - 1/50000000 s
    x = 10/1000 s (10 ms)
    x = (10/1000 s) / (1/50000000 s) = 10/1000 * 50000000 = 500000
    1 cs (setna sekundy) zmienia się co 500_000 taktów zegara */
    counter_modulo_k #(500_000) counter(.clk(clock), .aclr(1'b1), .enable(1'b1), .aload(1'b0), .rollover(clock_rollover));
    assign slow_clock = ~clock_rollover;

endmodule
```

Moduł N-bitowego licznika modulo k

```
module counter_modulo_k
#(parameter k = 20) ( // okres licznika
    input clk, aclr, enable, aload,
    input [N-1:0] data,
    output reg [N-1:0] Q,
    output rollover);

    function integer clogb2(input [31:0] v);
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)
            v = v >> 1;
    endfunction

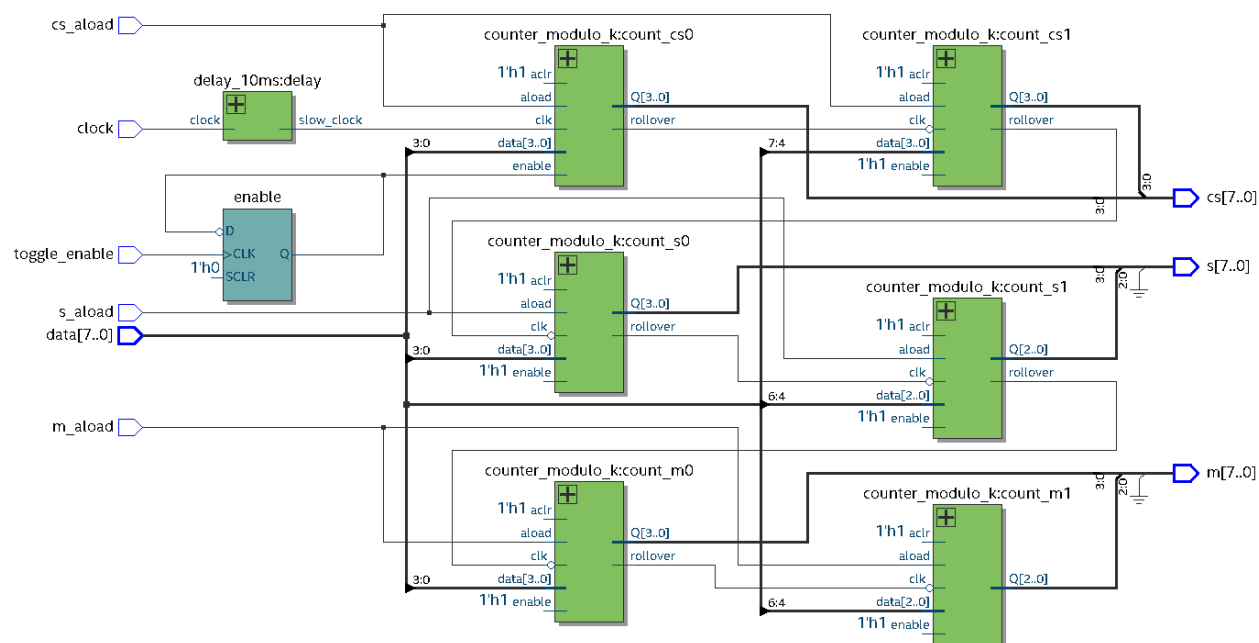
    localparam N = clogb2(k-1); // długość licznika w bitach

    initial Q = {N{1'b0}};
    always @ (posedge clk, negedge aclr, posedge aload)
        if (!aclr) Q <= {N{1'b0}};
        else if (aload) Q <= data;
        else begin
            // zanim sprawdzimy, czy Q == k-1 i ewentualnie wyzerujemy licznik, sprawdzamy, czy na wejściu zezwalającym jest 1
            if (enable) begin
                if (Q == k-1) Q <= {N{1'b0}};
                else Q <= Q + 1'b1;
            end
            else Q <= Q;
        end

    assign rollover = (Q == k-1);

endmodule
```

Wyniki syntezy

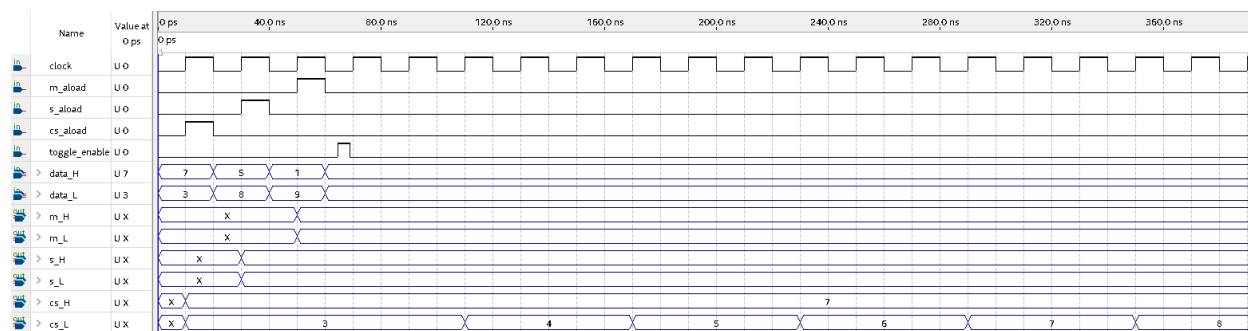


Wyniki symulacji funkcjonalnej

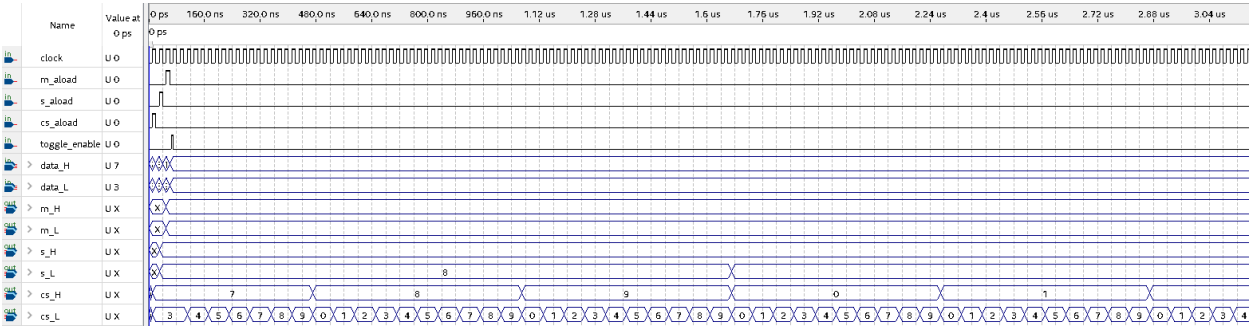
Aby w symulacjach było widać działanie zegara czasu rzeczywistego, parametryzujemy moduł opóźnienia delay_10ms jako licznik modulo 3. Wówczas wartość zegara czasu rzeczywistego zwiększa się o 1 setną sekundy (centysekundę – cs) co 3 zbocza narastające zegara „clock”.

Wprowadzanie danych

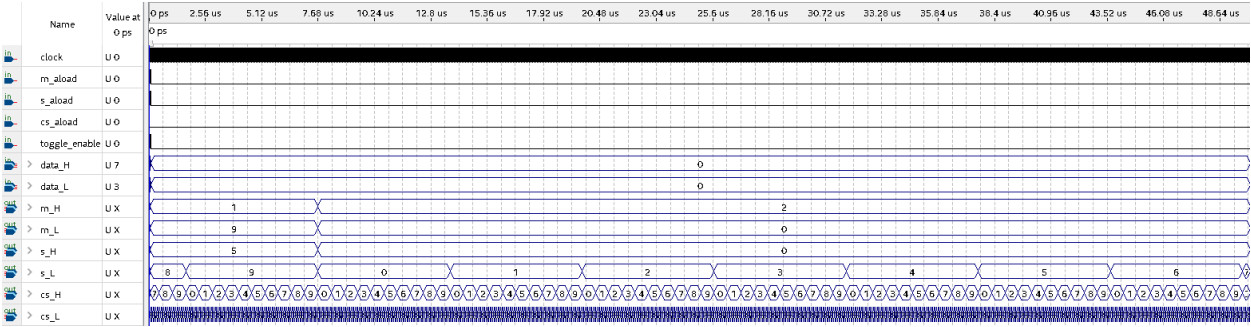
Zboczem narastającym na wejściach „cs_aload”, „s_aload”, „m_aload” wprowadzamy odpowiednio setne sekundy, sekundy, minuty obecne na wejściu „data”. Zboczem narastającym na wejściu „toggle_enable” zatrzymujemy lub wznowiamy zegar czasu rzeczywistego.



Odliczanie w skali setnych sekundy i sekund



Odliczanie w skali sekund i minut



Zadanie 4

Utwórz timer odliczający dwa z poniższych interwałów czasowych do sterowania miganiem diod LEDR[7:0] (wybrane przez prowadzącego).

- 0,1 sekundy (0,1 sek dioda zapalona, 0,1 sek. dioda zgaszona),
- 0,25 sekundy
- 0,5 sekundy
- 0,8 sekundy
- 1,2 sekundy
- 1,5 sekundy
- 1,8 sekundy
- 2 sekundy

Kod w Verilog

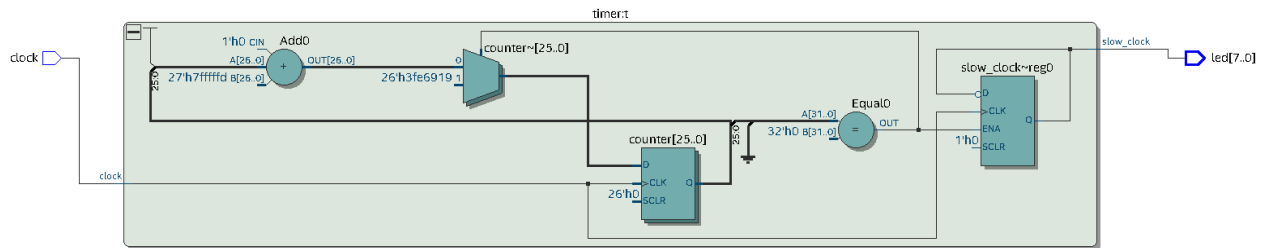
Moduł najwyższego poziomu z przyporządkowaniem do pinów płyty. Zegar CLOCK_50 ma częstotliwość 50 MHz, więc podajemy parametr timera HALF_CYCLE = 40 000 000, aby sygnał „leds_on” przez 0.8 s był 1, a następnie przez 0.8 s był 0.

```
module led_blink_on_board(  
    input CLOCK_50,  
    output [7:0] LEDR);  
  
    /* 1 - 1/50000000 s  
    x - 0.8 s  
    x = (0.8 s) / (1/50000000 s) = 0.8 * 50000000 = 40000000 */  
    wire leds_on;  
    timer #(40_000_000) t(CLOCK_50, leds_on);  
  
    assign LEDR = {8{leds_on}};  
  
endmodule
```

Moduł timera odliczający „HALF_CYCLE” taktów zegara „clock”

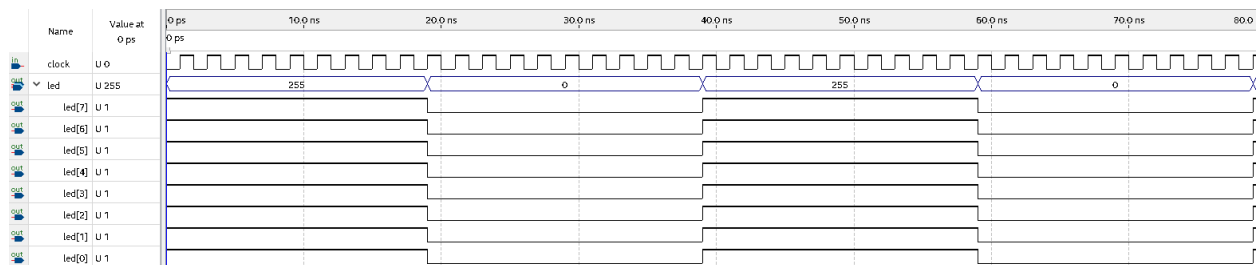
```
module timer  
    // liczba cykli zegara, po których slow_clock zmienia wartość na przeciwną  
    #(parameter HALF_CYCLE = 50_000_000)  
    (input clock,  
     output reg slow_clock);  
  
    function integer clogb2(input [31:0] v);  
        for (clogb2 = 0; v > 0; clogb2 = clogb2 + 1)  
            v = v >> 1;  
    endfunction  
  
    localparam N = clogb2(HALF_CYCLE - 1);  
  
    initial slow_clock = 1'b1;  
    reg [N-1:0] counter = HALF_CYCLE - 1;  
    always @ (posedge clock)  
        if (counter == 0) begin  
            counter <= HALF_CYCLE - 1;  
            slow_clock <= ~slow_clock;  
        end  
        else counter <= counter - 1;  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej

Aby w symulacji było widać działanie układu, parametryzujemy moduł timera jako odliczający 10 taktów (od 9 do 0) zegara „clock”. Wówczas sygnały „led” zmieniają wartość na przeciwną co 10 zbocz narastających zegara „clock”.



Wnioski

Wszystkie zadania zostały zrealizowane.